

P1110R0

A placeholder with no name

Published Proposal, 7 June 2018

This version:

<http://wg21.link/P1110R0>

Authors:

[Jeffrey Yasskin](#) (Google)

[JF Bastien](#) (Apple)

Audience:

EWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

<https://github.com/jyasskin/cxx-unnamed-placeholder/blob/master/unnamed.bs>

Abstract

A C++ token for locations that would normally introduce a name, that says not to introduce the name.

Table of Contents

- 1 Introduction and motivation**
- 2 Examples**
 - 2.1 Variables
 - 2.2 Structured Bindings
 - 2.3 Enumerators
 - 2.4 Speculative: Concept-constrained declarations
 - 2.5 Less useful examples
 - 2.5.1 Class, Struct, Enum

- 2.5.2 Unnamed parameters
- 2.5.3 Function names
- 2.5.4 Using
- 2.5.5 Concept introduction
- 2.6 Discouraged examples
 - 2.6.1 Anonymous Namespaces
 - 2.6.2 Anonymous Unions

3 Prior Art

- 3.1 Haskell
- 3.2 Python
- 3.3 Scala
 - 3.3.1 Pattern matching
 - 3.3.2 Defaulted definitions
 - 3.3.3 Imports
 - 3.3.4 Wildcards
- 3.4 Rust
- 3.5 C#
- 3.6 Java
- 3.7 Googlemock
- 3.8 Halide
- 3.9 Others

4 Spelling

- 4.1 
- 4.2 
- 4.3 
- 4.4 Repeated 
- 4.5 
- 4.6 Omit the name
- 4.7 Various Unicode characters

5 Semantic questions

- 5.1 Only on second use

6 Wording

7 Acknowledgements

References

Informative References

“I’ve been through the compiler on a var with no name
It felt good to be out of `main`
In the compiler you can remember your name
‘Cause there ain’t no diagnostic for to give you no pain”
— A Var with no Name

§ 1. Introduction and motivation

We have several locations in C++ where a name is expected, but programmers may not need to use that name ever again. Right now, they have to pick a different name for each location, and readers can’t immediately tell that the name isn’t used without an attribute or a naming convention. It would be nice to have a clear, enforceable way to state that a name is ignored.

To enforce that the name is actually ignored, and that people reading the code don’t think that an unused variable is actually used, using it in an expression should be an error instead of having a tag type like `nullptr_t`.

One of the authors of this paper filed [\[EWG35\]](#) on this subject years ago and then forgot about it. Unlike that issue, this paper does not expect the semantics of the placeholder name to be defined in terms of a generated unique name.

§ 2. Examples

The goal of this section is to provide an example of using the placeholder in all declaration contexts where it makes sense. These examples use `__` as the placeholder, but see [§4 Spelling](#) for an analysis of several options.

§ 2.1. Variables

To lock a mutex for a scope, without worrying that the variable name is already taken:

```
std::lock_guard<std::mutex> __ (a_mutex);
std::lock_guard<std::mutex> __ (a_second_mutex);
```

To hold some data alive that's used via internal pointers:

```
std::unique_ptr<T> ptr = ...;
auto& field1 = ptr->field1;
auto& field2 = ptr->field2
[__=std::move(ptr), &field1=field1, &field2=field2]() {...};
```

Note that the lambda capture case syntactically allows `[&__=init, ...](){};`, but this is probably not useful.

Classes can also have data members named `__`, which have the marginal use of reserving space for the future addition of new ABI-compatible fields. These fields cannot be used for the same purpose as in lambda captures because they can't be initialized.

```
class Foo {
    Foo(std::unique_ptr<T> ptr)
        : __ (std::move(ptr)) // Doesn't work.
    {}

    std::unique_ptr<T> __; // Not effective.
    int32_t __[4]; // Reserve 4 words of space for future fields.
};
```

§ 2.2. Structured Bindings

[\[P0144R2\]](#) §3.8 mentions that structured bindings could potentially use a syntax to ignore some fields from a bound structure:

```
tuple<T1,T2,T3> f();
auto [x, std::ignore, z] = f(); // NOT proposed: ignore second element
```

[\[P0144R2\]](#) suggests waiting until a pattern matching proposal, at which point the right token will fall out. I suggest that this proposal also provides a reasonable token, making this:

```
tuple<T1,T2,T3> f();
auto [x, __, z] = f(); // Ignore second element.
```

§ 2.3. Enumerators

This makes it very slightly easier to skip a value:

```
enum MmapBits {
    Shared,
    Private,
    __,
    __,
    Fixed,
    Rename,
    ...
};
```

I suspect that this is less clear than explicitly assigning the values, so it would make sense to not support this.

§ 2.4. Speculative: Concept-constrained declarations

To define a deduced-type non-type template parameter where the type matches a concept but the type isn't given a name in the body of the class or function:

```
template<Integral __ N> class integral_constant { ... };
```

To declare a deduced-type parameter or local variable whose type is constrained by a concept:

```
Numeric multiplyAdd(Numeric __ x, Numeric __ y, Numeric __ z) {
    Numeric __ multiplied = x * y;
    return multiplied + z;
}
```

§ 2.5. Less useful examples

§ 2.5.1. Class, Struct, Enum

These aren't necessary because developers can simply omit the name, but might still be useful to simplify the declaration syntax:

```
class __ : Base { ... };  
struct __ : Base { ... };  
enum __ { Enumerator };
```

We couldn't actually require a name there for several more versions of C++, if ever, but local style guides could require it in order to simplify what they need to teach new C++ developers.

§ 2.5.2. Unnamed parameters

Like unused class names, parameter names can simply be omitted instead of needing a placeholder. Again, requiring an explicitly-unused name, instead of allowing it to be omitted, simplifies the declaration grammar.

```
class C : public Base {  
    void Func(int i, long __) override;  
};  
  
template<typename __>  
class UsedInTemplateTemplateParam { ... };
```

§ 2.5.3. Function names

Using `__` to name a function is a similar case to declaring a variable, except that function definitions don't have the kind of side-effects that variable definitions have.

```
void __ (int arg) { ... }  
void __ (long arg) { ... }
```

Since the use of `__` prevents code from referring to these functions after they're declared, the above code creates two independent functions rather than an overload set (although I can't think of a way to

distinguish the two).

Virtual functions can also be named `__`. Having an unnamed virtual function take up a vtable slot could be useful to preserve ABI compatibility when a future function is added, but that's a matter for platform ABIs rather than the standard. This kind of virtual function cannot be overridden, of course, since it can't be named again after its declaration.

§ 2.5.4. Using

To use a type name, without being able to use the alias. I don't think using a type name has side-effects, so there's probably not much point.

```
using __ = typename Base::value_type;
```

Similarly non-useful:

```
typedef typename Base::value_type __;  
namespace __ = std::ranges;
```

§ 2.5.5. Concept introduction

Like name aliases, a `requires` clause doesn't have side-effects, so doesn't need to be assigned to an unnamed concept.

```
concept __ = requires { ... };
```

§ 2.6. Discouraged examples

§ 2.6.1. Anonymous Namespaces

It would be possible to use `__` as the name of an unnamed namespace:

```
namespace __ { ... };
```

However, the meaning of an anonymous namespace isn't just the meaning of a namespace with a name you never use again. Instead, [**namespace.unnamed**] says,

An *unnamed-namespace-definition* behaves as if it were replaced by

```
inlineopt namespace unique { /* empty body */ }  
using namespace unique ;  
namespace unique { namespace-body }
```

Because of this divergence in meaning from the other places `__` can appear, I recommend disallowing it as a namespace name.

§ 2.6.2. Anonymous Unions

[**class.union.anon**] says that

```
struct MyStruct {  
    union {  
        int i;  
        double d;  
    };  
};
```

Makes the union's members' names available as direct members of `MyStruct`. Because of that extra behavior, I also don't think we should allow

```
struct MyStruct {  
    union __ {  
        int i;  
        double d;  
    } __;  
};
```

We could say that naming either the union or the variable allows `__` in the other position, or disallow `__` in either position.

§ 3. Prior Art

§ 3.1. Haskell

Haskell uses a `_` token to indicate unused things:

```
head (x:_) = x
tail (_:xs) = xs
```

Note that the `_` can be "initialized" multiple times, unlike a named but unused variable:

```
data Colour = Colour { red::Int, green::Int, blue::Int, opacity::Int}

isOpaqueColour :: Colour -> Bool
isOpaqueColour (Colour _ _ _ opacity) = opacity == 255
```

§ 3.2. Python

`_` is conventionally used to name unused variables. For example:

```
label, has_label, _ = text.partition(':')
for _ in range(10):
    call_ten_times()
```

<https://stackoverflow.com/a/5893946/943619> describes this use, but there is no mention in the official Python documentation.

§ 3.3. Scala

Scala uses `_` for many kinds of placeholders:

§ 3.3.1. Pattern matching

```
def showImportantNotification(notification: Notification, importantPeopleInfo: List[String]) {
  notification match {
    case Email(email, _, _) if importantPeopleInfo.contains(email) =>
      "You got an email from special someone!"
    case SMS(number, _) if importantPeopleInfo.contains(number) =>
      "You got an SMS from special someone!"
    case other =>
      showNotification(other) // nothing special, delegate to our original
  }
}
```

Variable declarations can be patterns as well, which has the side-effect that

```
var _ = ...
```

defines an un-named variable like the subject of this proposal. This is less useful in Scala than in C++ because Scala doesn't have destructors.

§ 3.3.2. Defaulted definitions

The following initializes `x` to a default value depending on its type. For example, integral types get `0`, and reference types get `null`.

```
var x: T = _
```

§ 3.3.3. Imports

The import clause `import p._` ... makes available without qualification all members of `p` (this is analogous to `import p.*` in Java).

§ 3.3.4. Wildcards

Scala uses the `_` for wildcards in both types and functions. It's possible to write an anonymous function [just by using an `\_` in an expression](#). The first column in each of the rows in the following table is equivalent to the second.

<code>_ + 1</code>	<code>x => x + 1</code>
<code>_ * _</code>	<code>(x1, x2) => x1 * x2</code>
<code>(_: Int) * 2</code>	<code>(x: Int) => (x: Int) * 2</code>
<code>if (_) x else y</code>	<code>z => if (z) x else y</code>
<code>_.map(f)</code>	<code>x => x.map(f)</code>
<code>_.map(_ + 1)</code>	<code>x => x.map(y => y + 1)</code>

Similarly, [the following two types are equivalent](#):

```
Ref[_ <: java.lang.Number]
Ref[T] forSome { type T <: java.lang.Number }
```

§ 3.4. Rust

Rust provides a couple ways to [ignore values in patterns](#), which include variable and function parameter declarations:

`_:`

```
fn foo(_: i32, y: i32) {
    println!("This code only uses the y parameter: {}", y);
}
```

Variables prefixed with `_`:

The compiler will usually give an error for an unused variable, but if it's prefixed with `_`, the error is suppressed:

```
let s = Some(String::from("Hello!"));

if let Some(_s) = s {
    println!("found a string");
}
```

However, because of Rust's borrow checker, this is semantically different from a simple `_`: the `_s` takes ownership of the object it's bound to, where a `_` wouldn't.

`..` to ignore several parts of a value:

```
let numbers = (2, 4, 8, 16, 32);
match numbers {
    (first, .., last) => {
        println!("Some numbers: {}, {}", first, last);
    },
}
```

§ 3.5. C#

C# defines the `_` variable as a "[discard](#)", and allows their use in assignments in the following contexts:

- Tuple and object deconstruction.
- Pattern matching with `is` and `switch`.
- Calls to methods with `out` parameters.
- A standalone `_` when no `_` is in scope.

C# had to deal with `_` being a pre-existing valid variable name, which causes [certain uses of _ to be errors or bugs](#) if a `_` variable already exists in the same scope.

§ 3.6. Java

Java uses a `?` token to represent a "wildcard" generic argument.

```
void printCollection(Collection<?> c) {
    for (Object e : c) {
        System.out.println(e);
    }
}
```

Java generic arguments usually need to be declared, but wildcards imply that a function is generic:

```
class Collections {
    public static <T, S extends T> void copy(List<T> dest, List<S> src) {
        ...
    }

    class Collections {
        public static <T> void copy(List<T> dest, List<? extends T> src) {
            ...
        }
    }
}
```

§ 3.7. Googlemock

Googlemock defines `__` to match anything. It doesn't use it for declarations, but it's still an "ignore this" token.

```
// Expects the turtle to move forward by 100 units.
EXPECT_CALL(turtle, Forward(100));

// Expects the turtle to move forward.
EXPECT_CALL(turtle, Forward(__));
```

§ 3.8. Halide

From Halide's [documentation](#):

For example, consider the definition:

```
Func f, g;  
Var x, y;  
f(x, y) = 3;
```

A call to `f` with the placeholder symbol `_` will have implicit arguments injected automatically, so `f(2, _)` is equivalent to `f(2, _0)`, where `_0 = Var::implicit(0)`, and `f(_)` (and indeed `f` when cast to an Expr) is equivalent to `f(_0, _1)`. The following definitions are all equivalent, differing only in the variable names.

```
g(_) = f*3;  
g(_) = f(_) * 3;  
g(x, _) = f(x, _) * 3;  
g(x, y) = f(x, y) * 3;
```

§ 3.9. Others

I'm told that Swift, Erlang, Elixir, Prolog, OCaml, and F# also use the `_` for purposes related to ignoring variables, but I ran out of time to investigate and describe them all.

§ 4. Spelling

How should we spell the token?

§ 4.1. `_`

I believe `_` is the ideal spelling, but `[lex.name]` ¶3.2 only reserves it in the global namespace, and it's now used by libraries:

- [testing::_ in Googlemock](#)
- [Halide::_ in Halide](#)
- [The _\(\) macro in Gnu Gettext](#)

To avoid making these existing uses of `_` ill-formed, see [§5.1 Only on second use](#).

§ 4.2. `__`

This is already a reserved identifier per `[lex.name]` ¶3.1.

Each identifier that contains a double underscore `__` or begins with an underscore followed by an uppercase letter is reserved to the implementation for any use.

It takes a little longer to type than `_`, and in many fonts the difference between `__` and `_` may be unclear.

This paper uses `__` in the [examples](#).

The `__` identifier is used in some codebases, such as [the V8 JavaScript engine](#). Using a macro named `__` as V8 does means that breakage will only occur if that codebase also attempts to use this new feature.

§ 4.3. `?`

`?` might be confusable with the ternary `?:` operator, but developers can distinguish based on it appearing in a declaration rather than expression context.

`?` may also be confused with a wildcard like `*`, to mean "union all the possible values in this position", for example in a `nested::name::specifier`. Fortunately, I don't see a reason for `?` to be valid in contexts where that interpretation is plausible.

Using `?` for the placeholder identifier might prevent its use in the syntax for a forwarding reference. e.g. `ForwardingRef&&?`.

§ 4.4. Repeated `?`

Multiple `?` could be used to disambiguate some of the issues with a single `?`. Using `??` or `???` is more typing but is fairly obvious, though it would likely be better to make this a token so whitespace cannot be used between question marks.

§ 4.5. `auto`

The "attribute" proposal for concept declarations uses `auto` in a place a type might appear, which one might consider precedent for using it as a more general placeholder.

§ 4.6. Omit the name

We could just allow omitted names in more places:

```
auto = std::lock_guard(mx);
```

This option «mov[es] closer to "all syntax is valid and means something, but maybe not what you meant".» — [Tony Van Eerd](#)

§ 4.7. Various Unicode characters

We could use various Unicode characters, such as the replacement character , the empty set $\emptyset$, the interrobang , a trash can , one of the recycle signs , and countless other characters. These all have the shortcoming that not all toolchains and editors support them well or uniformly.

§ 5. Semantic questions

§ 5.1. Only on second use

We could declare that our placeholder makes variables anonymous only once it's used for the second time in a scope. That is:

```
int _ = 10; // fine
_ = 11; // fine, can use it
int _ = 12; // another _, fine
// at this point, both _ variables exist, but can no longer be accessed:
_ = 13; // error - which one?
```

Within this option, we could decide whether or not a second declaration is ill-formed if `_` has already been used within the scope.

Allowing this option at namespace scope, as [Googlemock](#) would need, seems to make ODR violations more likely. A file with only 1 unused variable (perhaps used to register a static initializer) would be likely to forget that variable's `static` and so generate an external symbol, which would collide with a similar symbol from another file.

§ 6. Wording

TBD

§ 7. Acknowledgements

Thanks to:

- Daveed Vandevoorde for suggesting using `?`.
- Thomas Köppe for suggesting `auto`.
- Mathias Stearn for suggesting omitting the name.
- Tony Van Eerd for providing an argument against omitting the name, and for suggesting that `_` only become magic when it's re-used.
- Richard Smith for pointing out Halide.
- Michael Park for a long list of languages that use `_`.

§ References

§ Informative References

[EWG35]

Jeffrey Yasskin. [\[tiny\] Some concise way to generate a unique, unused variable name](https://wg21.link/ewg35). Open.
URL: <https://wg21.link/ewg35>

[P0144R2]

Herb Sutter. [Structured Bindings](https://wg21.link/p0144r2). 16 March 2016. URL: <https://wg21.link/p0144r2>